

A Comparative Evaluation Of Transformer-Based Language Models

For the generation of Structured Algorithmic Representations such as Pseudocode and Executable Flow Diagrams from Natural Language Algorithm Specifications.

Building and Evaluating a Multi-Stage Pipeline: NL → Pseudocode → AST → Flow Diagram → Python → Test

SCAN ME!



Alan Johnson - 24038155

MSc Computer Science

Mentor: Beran Necat

July 2026

View Project Repository

PROJECT OVERVIEW

This project investigates the ability of transformer-based language models to convert natural language algorithmic descriptions into executable Python programs using a multi-stage, compiler-inspired pipeline.

The system transforms English Natural language (NL) descriptions into structured pseudocode, and builds an Abstract Syntax Tree (AST), then it generates a flow diagram (Control Flow Graph), from which it produces Python code, and finally tests the program for correctness.

MOTIVATION

- Bridging the gap between human problem descriptions and executable code.
- Improving automation in education, software development, and AI-assisted programming.
- Analysing the strengths and limitations of large language models in structured program generation.
- Improving semantic correctness between compiled programs from various languages.

RESEARCH OBJECTIVES

- Design and implement a multi-stage pipeline that converts language algorithmic descriptions into executable Python programs.
- Define a structured pseudocode language and develop a grammar using the Cambridge Pseudocode Guide for Teachers as a baseline for comparison
- Generate program flow diagrams (Control Flow Graphs) from the AST generated from the Pseudocode.
- Generate Python code from the flow diagram.
- Develop an automated testing framework to evaluate correctness.
- Evaluate and compare multiple transformer-based models across all stages of the pipeline.

KEY CONTRIBUTIONS

- A complete end-to-end program generation pipeline.
- A benchmark dataset of natural language algorithm descriptions with test cases.
- A benchmark dataset of pseudocode algorithm descriptions with test cases.
- A framework for evaluating LLMs in multi-stage program generation.
- Insights into error propagation and failure patterns across stages.

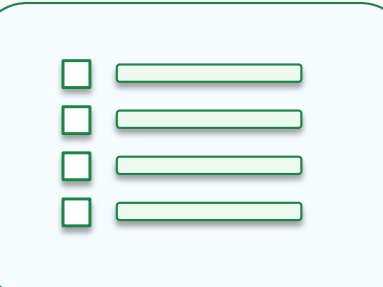
METHODOLOGY: MULTI-STAGE PROGRAM SYNTHESIS PIPELINE

1. ENGLISH NL DESCRIPTION



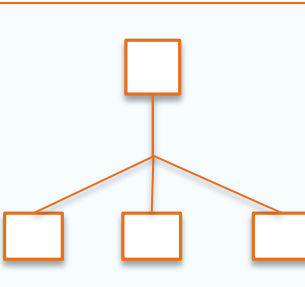
Natural language algorithm specification

2. PSEUDOCODE GENERATION



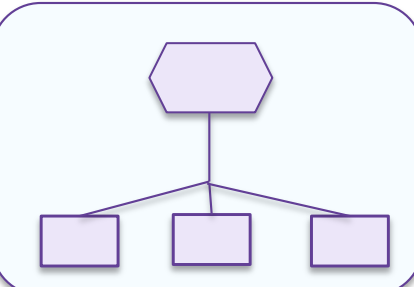
Transformer model generates structured pseudocode

3. AST GENERATION



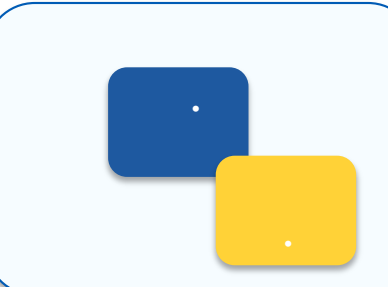
Parser converts pseudocode into an Abstract Syntax Tree

4. FLOW DIAGRAM GENERATION



AST is converted into a Control Flow Graph (Flow Diagram)

5. PYTHON CODE GENERATION



Python code is generated from the flow diagram

6. TESTING & EVALUATION



Code is executed against test cases and evaluated

EXAMPLE PIPELINE WALKTHROUGH

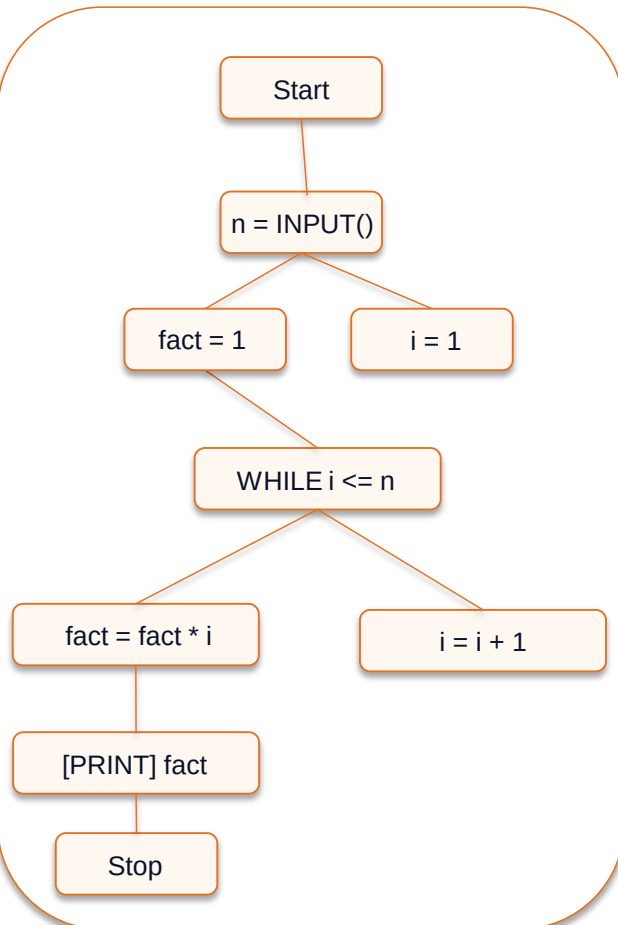
ENGLISH NL DESCRIPTION

Find the factorial of a number using an iterative approach.

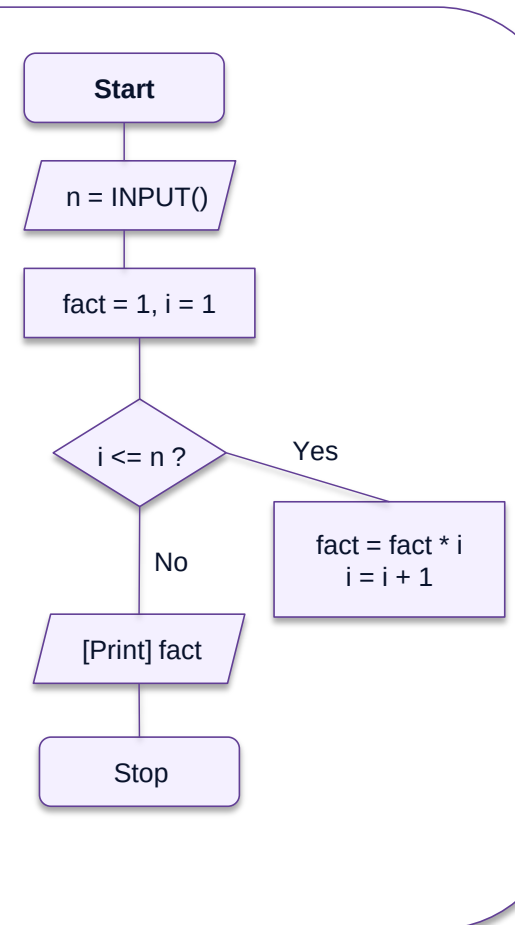
PSEUDOCODE

```
1. Start
2. n <- INPUT()
3. fact <- 1
4. i <- 1
5. WHILE i <= n DO
  5.1 fact <- fact * i
  5.2 i <- i + 1
6. ENDWHILE
7. OUTPUT fact
8. STOP
```

AST



FLOW DIAGRAM (CFG)



PYTHON CODE

```
n = int(input())
fact = 1
i = 1
while i <= n:
    fact = fact * i
    i = i + 1
print(fact)
```

TEST RESULT

Input: 5
Expected Output: 120
Actual Output: 120
Result: ✓ PASS

METHODOLOGY SUMMARY

- Experimental and comparative research design
- Multi-stage pipeline implementation
- Quantitative evaluation at each stage
- Qualitative error analysis
- Statistical comparison between models

EXPECTED OUTCOMES

- A working end-to-end program generation system
- A Comprehensive evaluation of LLM capabilities
- An Identification of common error types and where they occur
- An Analysis of the accuracy of program generation from Flow Diagrams (CFD)
- Insights for improving future model-based code generation systems
- a metric for possible language to language compiled code

APPLICATIONS

- AI-assisted programming tools
- Automated tutoring and education
- Rapid prototyping from requirements
- Documentation and flow automation
- Supporting non-programmers in algorithm design

DATA & EVALUATION

DATASET

- 50 algorithmic problems
- Increasing difficulty
- Includes input/output test cases
- Covers varying complexity (arithmetic, conditionals, loops, searching, sorting, etc.)

EVALUATION METRICS

- Pseudocode Syntax Validity
- AST Parsing Success Rate
- Flow Diagram Structural Validity
- Python Code Execution Success
- Output Correctness

AI MODELS TO BE EVALUATED

- Gwen 3.6 plus
- Groq llama-3.3-70b-versatile
- Gemini 3 flash
- DeepSeek
- Claude sonnet 4.6
- ChatGPT 5.5

TECHNOLOGIES & TOOLS

- Python
- PyTorch / Transformers (Hugging Face)
- NetworkX / Graphviz (Flow Diagrams)
- Git & GitHub
- PyCharm
- PyTest (Testing Framework)

CONTACT

- Student: Alan Johnson
- Email: y3x82@students.keele.ac.uk
- GitHub: https://github.com/ajohnson-2210/Final_Project
- University of Keele
- Department of Computer Science

SHORT BIOGRAPHY

I first began thinking about using control flow diagrams to generate code early in my computer science journey. At the time, I was tired of repeatedly typing the same boilerplate for web pages especially the header section and started imagining a drag and drop tool that would let me place HTML elements directly onto a page. This was before React became widespread; in fact, I discovered React halfway through the project and adopted it immediately. While planning that tool, another idea emerged: *What if a program's flow diagram could be dropped into an application and automatically compiled into code?* I was thinking mainly about JavaScript then, but the concept was broader than any single language. I didn't at that time have the skill set to build such a system, but the idea stayed with me.

The appeal is simple; a flow diagram is a blueprint for a program. If the logic is expressed correctly, it should be possible to compile that diagram into any programming language, producing equivalent behaviour regardless of language syntax. In theory, this would eliminate the need to manually ensure semantic alignment when translating code between languages, because the diagram itself defines the logic unambiguously.